CSCE 322 – Final Exam Practice Worksheet Solutions
Final Exam Practice – Spring 2026

Note. These solutions follow the grammars and big-step semantics in the language reference. Environment extensions are written with the newest binding at the *front*. We use σ for environments, h for the heap, and $\ell_1, \ell_2, \ldots$ for locations. Locations are never reused.

Problem 1 – CFG Design and Ambiguity

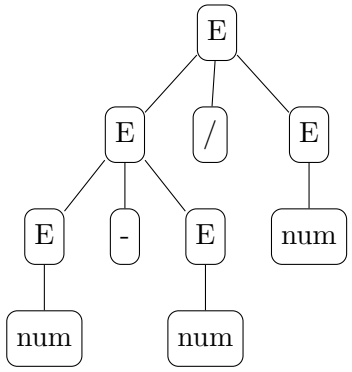
(a) **Language of G.**

$$L(G) = \{\, a^n\, b^m\, c^n \mid n \geq 0,\ m \geq 0 \,\}.$$

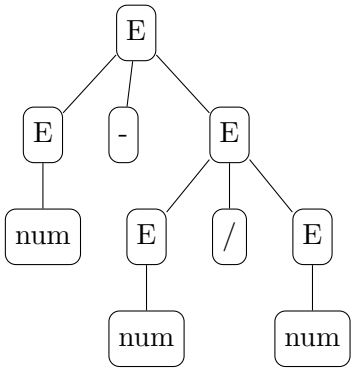
S generates an outer $a^n \ldots c^n$ envelope (any $n \geq 0$); T fills the inside with any number of b's (any $m \geq 0$).

(b) **Two parse trees for `num - num / num`.**

Tree 1: $(\texttt{num} - \texttt{num}) \,/\, \texttt{num}$.



Tree 2: $\texttt{num} - (\texttt{num} \,/\, \texttt{num})$.



Both trees derive the same string under different groupings, which proves the grammar is ambiguous (a single string admits two distinct parse trees).

(c) **CFG for $L = \{a^n b^{2n} \mid n \geq 1\} \cup \{c^k d^k \mid k \geq 0\}$:**

$$
\begin{aligned}
S &\to A \mid C \\
A &\to a\,A\,b\,b \mid a\,b\,b \\
C &\to c\,C\,d \mid \varepsilon
\end{aligned}
$$

A generates $a^n b^{2n}$ for $n \geq 1$ (each rewrite adds one a on the left and two b's on the right; the base case is $a\,b\,b$). C generates $c^k d^k$ for $k \geq 0$.

Leftmost derivation for `aabbbb`:

$$
\begin{aligned}
S &\Rightarrow A \\
&\Rightarrow a\,A\,b\,b \\
&\Rightarrow a\,a\,b\,b\,b\,b.
\end{aligned}
$$

Problem 2 – Free/Bound Variables and α-Conversion

(a)

$$(\lambda x.\, \lambda y.\, x\ y\ (\lambda x.\, x))\ (\lambda y.\, y\ z)$$

FV: $\{z\}$.
BV: $\{x, y\}$, with x at two binders and y at two binders.
α-conversion (rename inner λx and the argument's λy):

$$(\lambda x.\, \lambda y.\, x\ y\ (\lambda x_1.\, x_1))\ (\lambda y_1.\, y_1\ z)$$

Why: required so that β-reducing the application does not capture the binders' x or y – after substitution every binder must remain distinct.

(b)

$$\lambda p.\, \big(\lambda q.\, p\ (\lambda p.\, q\ (\lambda q.\, p\ q))\big)$$

FV: $\emptyset$.
BV: $\{p, q\}$, with p at two binders and q at two binders.
α-conversion (rename inner λp and innermost λq):

$$\lambda p.\, \big(\lambda q.\, p\ (\lambda p_1.\, q\ (\lambda q_1.\, p_1\ q_1))\big)$$

Why: both p and q are reused as binders; renaming makes scope unambiguous so each occurrence resolves to a single, unique binder.

(c)

$$\lambda f.\, \big((\lambda g.\, f\ (g\ f))\ (\lambda f.\, \lambda x.\, f\ (f\ x))\big)$$

FV: $\emptyset$.
BV: $\{f, g, x\}$, with f at two binders.
α-conversion (rename the argument's λf):

$$\lambda f.\, \big((\lambda g.\, f\ (g\ f))\ (\lambda f_1.\, \lambda x.\, f_1\ (f_1\ x))\big)$$

Why: the outer f is referenced inside $(g\ f)$ while the argument has its own local f; renaming prevents a future substitution from confusing the two.

Problem 3 – Church Encodings and β-Reduction

(a) Reduction of $\texttt{head}(\texttt{tail}(\texttt{tail}(\texttt{cons}\ p\ (\texttt{cons}\ q\ (\texttt{cons}\ r\ \texttt{nil})))))$.

Unfold the `cons`'s into pairs:

$$\texttt{cons}\ p\ (\texttt{cons}\ q\ (\texttt{cons}\ r\ \texttt{nil})) \to_\beta \texttt{pair}\ p\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil})).$$

Compute the first `tail` in detail (each `tail` peels the outer pair's first component):

$$\begin{aligned}
\texttt{tail}\ (\texttt{pair}\ p\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil}))) &= (\lambda p'.\ p'\ (\lambda x.\ \lambda y.\ y))\ (\texttt{pair}\ p\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil}))) \\
&\to_\beta (\texttt{pair}\ p\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil})))\ (\lambda x.\ \lambda y.\ y) \\
&\to_\beta (\lambda y.\ \lambda f.\ f\ p\ y)\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil}))\ (\lambda x.\ \lambda y.\ y) \\
&\to_\beta (\lambda f.\ f\ p\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil})))\ (\lambda x.\ \lambda y.\ y) \\
&\to_\beta (\lambda x.\ \lambda y.\ y)\ p\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil})) \\
&\to_\beta (\lambda y.\ y)\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil})) \\
&\to_\beta \texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil}).
\end{aligned}$$

The second `tail` follows the same pattern:

$$\texttt{tail}\ (\texttt{pair}\ q\ (\texttt{pair}\ r\ \texttt{nil})) \to_\beta^* \texttt{pair}\ r\ \texttt{nil}.$$

Finally, $\texttt{head}\ (\texttt{pair}\ r\ \texttt{nil}) = \texttt{fst}\ (\texttt{pair}\ r\ \texttt{nil}) \to_\beta^* r$.

Final value: r.

(b) Reduction of $\texttt{head}(\texttt{tail}(\texttt{cons}\ (\texttt{or}\ \texttt{true}\ \texttt{false})\ (\texttt{cons}\ (\texttt{and}\ (\texttt{not}\ \texttt{false})\ \texttt{true})\ \texttt{nil})))$.

Unfold the two `cons`'s, then apply $\texttt{tail} = \texttt{snd}$ and $\texttt{head} = \texttt{fst}$ (same pattern as part (a)) to extract the second list element:

$$\texttt{head}\big(\texttt{tail}(\ldots)\big) \to_\beta^* \texttt{and}\ (\texttt{not}\ \texttt{false})\ \texttt{true}.$$

Reduce the inner `not false`:

$$\begin{aligned}
(\lambda b.\ b\ \texttt{false}\ \texttt{true})\ \texttt{false} &\to_\beta \texttt{false}\ \texttt{false}\ \texttt{true} \\
&= (\lambda t.\ \lambda f.\ f)\ \texttt{false}\ \texttt{true} \to_\beta (\lambda f.\ f)\ \texttt{true} \to_\beta \texttt{true}.
\end{aligned}$$

So the expression becomes $\texttt{and}\ \texttt{true}\ \texttt{true}$. Reduce using $\texttt{and} = \lambda p.\ \lambda q.\ p\ q\ \texttt{false}$:

$$\begin{aligned}
(\lambda p.\ \lambda q.\ p\ q\ \texttt{false})\ \texttt{true}\ \texttt{true} &\to_\beta (\lambda q.\ \texttt{true}\ q\ \texttt{false})\ \texttt{true} \\
&\to_\beta \texttt{true}\ \texttt{true}\ \texttt{false} \\
&= (\lambda t.\ \lambda f.\ t)\ \texttt{true}\ \texttt{false} \\
&\to_\beta (\lambda f.\ \texttt{true})\ \texttt{false} \\
&\to_\beta \texttt{true}.\ \checkmark
\end{aligned}$$

Final value: `true`.

Problem 4 – Canonical Prefix Evaluation

(a) $(+\ 1\ 2) \to 3$; $(-\ 5\ 1) \to 4$; $(*\ 3\ 4) \to 12$; $(*\ 2\ 3) \to 6$; $(-\ 12\ 6) \to 6$. **Result: 6.**

(b) (`lt` 4 4) → `false`; (`not false`) → `true`. $(*\ 2\ 3) \to 6$; (`eq` 6 6) → `true`; (`or true false`) → `true`. (`and true true`) → `true`, so take the then-branch: $(*\ 5\ 2) \to 10$; $(+\ 3\ 1) \to 4$; $(-\ 10\ 4) \to 6$. **Result: 6.**

(c) Type error. `or` evaluates its left operand first; ($*$ 3 `false`) requires two integers, but `false` is a boolean – evaluation gets stuck before `or` can short-circuit. Short-circuiting only helps when the left operand reduces to a definite boolean; here it produces no value at all.

(d) (i) Semantic (type) error. The expression matches `(and CCond CCond)` syntactically (*CCond* accepts arbitrary *Variable*-like forms in the relaxed grammar). Operationally, $(+\ 7\ 2) \to 9$, an integer, but `and` requires booleans, so evaluation gets stuck.

(ii) Two minimal one-edit fixes:

1. Replace + with `gt`: `(and (gt 5 3) (gt 7 2))`. (`gt` 5 3) → `true`; (`gt` 7 2) → `true`; (`and true true`) → **`true`**.
2. Replace + with `lt`: `(and (gt 5 3) (lt 7 2))`. (`gt` 5 3) → `true`; (`lt` 7 2) → `false`; (`and true false`) → **`false`**.

Problem 5 – Nested `var` and Environments

(a) Start with $\sigma_0 = [\, b \mapsto 2,\ a \mapsto 6\,]$.

- **Step 1.** $(-\ a\ b)$ under σ_0: $a = 6$, $b = 2$, value 4.

$$\sigma_1 = [\, a \mapsto 4,\ b \mapsto 2,\ a \mapsto 6\,].$$

- **Step 2.** $(+\ a\ b)$ under σ_1: leftmost $a = 4$, $b = 2$, value 6.

$$\sigma_2 = [\, b \mapsto 6,\ a \mapsto 4,\ b \mapsto 2,\ a \mapsto 6\,].$$

- **Step 3.** $(*\ a\ b)$ under σ_2: $a = 4$, $b = 6$, value 24.

$$\sigma_3 = [\, a \mapsto 24,\ b \mapsto 6,\ a \mapsto 4,\ b \mapsto 2,\ a \mapsto 6\,].$$

  Now evaluate the outer $(*\ E_1\ E_2)$ under σ_3.

- **Step 4 (left factor E_1):** `(var (b (- b a)) (+ a b))`.
  - $(-\ b\ a)$ under σ_3: $b = 6$, $a = 24$, value -18.
  - $\sigma_{3a} = [\, b \mapsto -18,\ a \mapsto 24,\ b \mapsto 6,\ a \mapsto 4,\ b \mapsto 2,\ a \mapsto 6\,]$.
  - $(+\ a\ b)$ under σ_{3a}: $a = 24$, $b = -18$, value 6.
  - Out of scope; back to σ_3.

- **Step 5 (right factor E_2):** `(var (a (+ a b)) (- a b))`.
  - $(+\ a\ b)$ under σ_3: $a = 24$, $b = 6$, value 30.
  - $\sigma_{3b} = [\, a \mapsto 30,\ a \mapsto 24,\ b \mapsto 6,\ a \mapsto 4,\ b \mapsto 2,\ a \mapsto 6\,]$.
  - $(-\ a\ b)$ under σ_{3b}: $a = 30$, $b = 6$, value 24.
  - Out of scope; back to σ_3.

- **Step 6.** $(*\ 6\ 24) = 144$.

Final value: 144.

(b) In $(+\ a\ b)$ (left factor): $a = 24$ (the step-3 binding) and $b = -18$ (the local `var` of the left factor, which shadows older b's). In $(-\ a\ b)$ (right factor): $a = 30$ (the local `var` of the right factor, which shadows the step-3 $a = 24$) and $b = 6$ (from step 2 – the right factor introduces no new b). Each inner `var` only shadows the variable it binds, so the other variable's lookup walks past the local frame to the next visible binding.

(c) If `(var (a ...) ...)` came before `(var (b ...) ...)` at the innermost level, the new a would be computed under σ_2 (not under σ_2 extended by the new b), and the new b would then see this new a. The chain shifts and the final answer changes – order matters because each binding's value is computed in the environment that exists *at that moment*.

Problem 6 – Closures, Higher-Order Functions, and Heap

(a) Start from $\sigma_0 = [\,]$ and $h_0 = [\,]$.

- **Step 1.** $(\texttt{ref}\ 1) \to \ell_1$, $h_1 = [\,\ell_1 \mapsto 1\,]$. $\sigma_1 = [\,s \mapsto \ell_1\,]$.
- **Step 2.** The outer `fun` builds the closure for `make-mult`, capturing σ_1:

$$\sigma_2 = [\,\texttt{make-mult} \mapsto \langle \ldots, \sigma_1 \rangle,\ s \mapsto \ell_1\,].$$

- **Step 3.** (`apply` (`make-mult` (0))) to bind `g`.
  - Captured env σ_1. Argument 0.
  - Call env (static): $[\,m \mapsto 0,\ s \mapsto \ell_1\,]$.
  - Inner `fun` returns $\mathit{closure}^{(g)}_{\texttt{mult}} = \langle \texttt{mult}, (n), (*\ n\ (\texttt{deref}\ s)), [\,m \mapsto 0,\ s \mapsto \ell_1\,] \rangle$. (Including `mult` in its own captured env is also acceptable; `mult` doesn't reference itself recursively.)
  - Bind `g`. $\sigma_3 = [\,g \mapsto \mathit{closure}^{(g)}_{\texttt{mult}},\ \texttt{make-mult} \mapsto \ldots,\ s \mapsto \ell_1\,]$.
- **Step 4.** `(var (u (wref s 7)) ...)`.
  - $(\texttt{wref}\ s\ 7)$: $s \mapsto \ell_1$, value 7, check $h_1(\ell_1) = 1 \neq \texttt{FREE}$, update $h_2 = [\,\ell_1 \mapsto 7\,]$, return 7.
  - $\sigma_4 = [\,u \mapsto 7,\ g \mapsto \ldots,\ \texttt{make-mult} \mapsto \ldots,\ s \mapsto \ell_1\,]$.
- **Step 5.** (`apply` (`make-mult` (0))) again to bind `h`. Same procedure as step 3: the captured env is still $[\,m \mapsto 0,\ s \mapsto \ell_1\,]$ – the location is the same ℓ_1 (no new heap cells are allocated; per the no-reuse rule we just don't allocate at all here). Call this closure $\mathit{closure}^{(h)}_{\texttt{mult}}$.

$$\sigma_5 = [\,h \mapsto \mathit{closure}^{(h)}_{\texttt{mult}},\ u \mapsto 7,\ g \mapsto \ldots,\ \texttt{make-mult} \mapsto \ldots,\ s \mapsto \ell_1\,].$$

- **Step 6.** (`apply` (g (4))) under σ_5, h_2.
  - g's captured env $[\,m \mapsto 0,\ s \mapsto \ell_1\,]$. Call env: $[\,n \mapsto 4,\ m \mapsto 0,\ s \mapsto \ell_1\,]$.
  - $(*\ n\ (\texttt{deref}\ s))$: $n = 4$; $(\texttt{deref}\ s)$ reads $h_2(\ell_1) = 7$; $(*\ 4\ 7) = 28$.
- **Step 7.** (`apply` (h (4))): same call shape, same heap read $\to 28$.
- **Step 8.** $(+\ 28\ 28) = 56$.

Final value: 56. Final heap: $h = [\,\ell_1 \mapsto 7\,]$**.**

(b) Both closures captured the same binding $s \mapsto \ell_1$ (a location), not the value stored there at the time of capture. When either closure body runs and evaluates $(\texttt{deref}\ s)$, it reads the heap at ℓ_1 – and both calls happen *after* $(\texttt{wref}\ s\ 7)$, so both see 7. Closures don't "remember" that s was 1 when they were defined, because environments map names to locations and the heap is what holds the actual mutable contents.

(c)

i. Evaluates to 0. $(\texttt{ref}\ 0) \to \ell_1$, $h(\ell_1) = 0$. $(\texttt{ref}\ p)$ evaluates p to the location ℓ_1 and allocates a fresh ℓ_2 with $h(\ell_2) = \ell_1$. $q \mapsto \ell_2$. $(\texttt{deref}\ q) = \ell_1$, then $(\texttt{deref}\ \ell_1) = 0$. Final $h = [\,\ell_1 \mapsto 0,\ \ell_2 \mapsto \ell_1\,]$.

ii. Evaluates to 16. (`ref` 3) $\to \ell_1$, $h(\ell_1) = 3$. (`wref` p 8) updates $h(\ell_1) = 8$ and *returns* 8, so $q = 8$. (`deref` p) $= 8$; (+ 8 8) $= 16$. Final $h = [\,\ell_1 \mapsto 8\,]$.

iii. Runtime (type) error at the application. The function is well-formed, but the body (`deref` x) requires x to be a location; at the call $x = 5$, an integer.

iv. Runtime error at (`wref` p 9). After (`free` p), $h(\ell_1) =$ `FREE`. (`wref` p 9) checks $h(\ell_1) \neq$ `FREE` and fails.

v. Syntax error at the inner `var`. The form `(var p (ref 5) ...)` has the binding tokens unparenthesized; the grammar requires `(var (Variable Expr) Expr)`.

Problem 7 – Conceptual Questions

(a) The proposal replaces lexical scope with dynamic scope: the function body would resolve free variables against whatever environment exists at the call site, not where the function was written. This breaks abstraction – a function's behavior would silently depend on caller-local variable names, so any rename of a caller's local could change the function's result. Example: `(fun ((f (x)) (+ x n)) (var (n 99) (apply (f (1)))))` should return $1 + n_{\text{def}}$ under closures, but under the proposal it returns 100, with the function silently picking up the caller's n.

(b) The environment maps variable names to values (or locations); `var` prepends bindings, shadows older ones, and bindings disappear with their scope – the binding itself is never mutated. The heap maps locations to values and supports in-place mutation via `wref` and invalidation via `free`. Variables follow lexical scope and disappear with their block, while heap cells outlive scopes and can be aliased through multiple variables – collapsing them into one map would either destroy aliasing (every variable would have its own cell) or pollute scope handling (heap cells would vanish when their introducing scope exits).

(c) A *syntax error* means the source string does not match the grammar – e.g., `(var x (ref 5) (deref x))` violates *VarExpr*, which requires a `(Variable Expr)` binding pair. A *semantic/runtime error* means the expression parses fine but evaluation gets stuck – e.g., `(deref 5)` parses as a valid *DerefExpr* but fails because `deref` requires a location, not an integer.

(d) Under `[App]` (static), `g`'s captured env has $k = 5$, so $(* \ 4 \ 5) = \mathbf{20}$. Under `[App_dyn]` (dynamic), the body runs in the caller's env where the most recent k is 2, so $(* \ 4 \ 2) = \mathbf{8}$. A language designer prefers static scope because a function's behavior depends only on its definition site – readers can reason about a function locally without tracing every call site, and refactoring local variables in unrelated callers cannot accidentally change the function's meaning.